

Introducing Celery for Python+Django

For background task processing and deferred execution in Python with Django, Web developers and system admins can try out Celery.



A synchronous mass email delivery, clickstreams like the number of hotels being watched or the number of likes, image resizing, video processing, connecting to third-party APIs, enhancing the user experience by executing tasks in the background rather than in the request-response loop, or executing a large number of queries on a database for reporting... these are some of the scenarios in which you might want to use distributed job queues in order to scale your application. Job queues also separate the tasks to execute in real-time from those to be scheduled for later. There are many use cases in which job queues help you achieve better user experiences. This article introduces readers to the use of Celery to leverage this in Python and Django applications.

Celery is based on distributed message-passing for asynchronous task queues/job queues. It is capable of supporting both real-time operations as well as scheduled jobs. The tasks can be run on a single worker or multiple workers concurrently, to take advantage of multi-processing. Celery provides a powerful and flexible interface for defining, executing, managing and monitoring tasks. A Celery system can consist of multiple workers and brokers, yielding high availability and horizontal scaling. Celery is suitable for applications that need to achieve any of the following:

1. Execute tasks asynchronously.
2. Distributed execution of expensive processes.
3. Third-party API usage.
4. Periodic/ scheduled tasks.

5. Retrying tasks.
6. Enhancing the user experience.

Celery architecture

Task queues are used to distribute work across workers. Celery task queues are based on the Advanced Message Queue Protocol. By default, it uses RabbitMQ as its message broker; however, users are not limited to RabbitMQ but can use Redis, MongoDB or Beanstalk too. Figure 1 depicts this process.

Step 1: The AMQP receives a task from the client, which may be a Web app or a Python program.

Step 2: The workers constantly monitor the queue; as soon as a message is dropped in the queue, one of the workers picks it up and executes it.

Step 3: Depending on the configuration, it may or may not save the result, once it has finished processing the execution.

Setting up Celery

Although the choice of message broker is entirely your decision, for this article, I assume we are using RabbitMQ (it's what I use in production, too). Before installing Celery, you must have RabbitMQ installed and running (start it with *rabbitmq-server start*). Then, all you need to install Celery is *pip install -U celery* and you're ready to create your first program using Celery.

Make a project folder, and in it, create a file *tasks.py*, which will contain the tasks you want to perform using Celery. Here's a sample program I'll be using to fetch JSON and read its contents: